

Projet Java — Sokoban : jeu et résolution algorithmique

Table des matières

1. Contexte	2
2. Exemple de grille	2
3. Partie A — Jouer à Sokoban	2
4. Partie B — Vérification de la victoire	3
5. Partie C — Plusieurs niveaux	3
6. Partie D — Résolution automatique	4
7. Partie E — Recherche en profondeur	4
8. Partie F — Recherche en largeur	4
9. Comparaison des algorithmes	4
10. Contraintes techniques	5
11. Méthodes attendues	5
12. Niveau standard	6
13. Niveau avancé	6
14. Fonctionnalités bonus	6
15. Exemple de menu final	7

1. Contexte

Sokoban est un jeu de réflexion se déroulant sur une grille. Le joueur doit déplacer un ensemble de caisses afin de les placer sur des cases objectifs. Le joueur peut se déplacer horizontalement ou verticalement. Il peut pousser une caisse lorsqu'une case libre se trouve derrière celle-ci, mais il ne peut jamais tirer une caisse.

L'objectif du projet est de développer une version console du jeu en Java, puis d'ajouter un module permettant au programme de rechercher automatiquement une solution.

Le projet doit mettre en œuvre les notions d'algorithmique étudiées : tableaux, méthodes, boucles, conditions, récursivité, recherche, piles/files et analyse de complexité.

2. Exemple de grille

Une grille peut être représentée par les caractères suivants :

- # : mur
- : case libre
- @ : joueur
- \$: caisse
- . : objectif
- * : caisse placée sur un objectif

Exemple :

```
#####  
#      #  
#  .  #  
#  $  #  
#  @  #  
#      #  
#####
```

Le joueur doit déplacer la caisse \$ vers l'objectif .

3. Partie A — Jouer à Sokoban

Le programme doit permettre à un joueur humain de jouer au clavier. À chaque tour, le programme affiche la grille et demande une commande :

- z** : haut
- s** : bas
- q** : gauche
- d** : droite
- x** : quitter

Les commandes invalides doivent être détectées. Le joueur ne doit pas pouvoir traverser un mur. Lorsqu'il rencontre une caisse, il peut la pousser uniquement si la case située derrière celle-ci est libre.

4. Partie B — Vérification de la victoire

Le programme doit déterminer automatiquement si toutes les caisses ont été placées sur des objectifs.

Exemple :

```
#####
#       #
#  *   #
#  @   #
#       #
#####
```

Victoire !

Nombre de mouvements : 27

Le programme doit également afficher le nombre de mouvements effectués.

5. Partie C — Plusieurs niveaux

Le programme doit proposer plusieurs niveaux.

Exemple :

===== SOKOBAN =====

1. Niveau 1

2. Niveau 2

3. Niveau 3

4. Quitter

Votre choix :

Les niveaux peuvent être stockés directement dans le programme sous forme de tableaux de caractères.

Un niveau doit contenir :

- **au moins un joueur ;**
- **au moins une caisse ;**
- **autant d'objectifs que de caisses ;**
- **des murs ;**
- **des cases libres.**

6. Partie D — Résolution automatique

La partie avancée du projet consiste à développer un solveur automatique. Le programme doit pouvoir répondre à la question :

Existe-t-il une solution permettant de placer toutes les caisses sur les objectifs ?

Le solveur doit explorer différents états possibles du jeu. Pour chaque état, le programme doit pouvoir déterminer :

- la position du joueur ;
- la position des caisses ;
- les mouvements possibles ;
- les états déjà explorés ;
- si l'état correspond à une victoire.

7. Partie E — Recherche en profondeur

Implémenter une première méthode de résolution utilisant une recherche en profondeur (Depth-First Search: DFS). Cette méthode peut être réalisée de manière récursive.

Le programme doit éviter de revisiter indéfiniment les mêmes états.

Vous devez expliquer les éléments suivants :

- le cas de base ;
- le cas récursif ;
- la condition d'arrêt ;
- les avantages et limites de cette approche.

8. Partie F — Recherche en largeur

Implémenter ensuite une recherche en largeur (Breadth-First Search: BFS).

Le principe est d'explorer les états par distance croissante depuis l'état initial. Le programme doit utiliser une structure de type file pour gérer les états à explorer. L'objectif est notamment de pouvoir trouver une solution courte.

9. Comparaison des algorithmes

Le programme devra permettre de comparer les deux méthodes.

Pour une même grille, afficher par exemple :

===== **COMPARAISON** =====

DFS

Solution trouvée : oui

Mouvements : 42

États explorés : 1837

BFS

Solution trouvée : oui

Mouvements : 18

États explorés : 1254

Les valeurs ci-dessus sont uniquement un exemple.

Vous devez expliquer pourquoi les deux algorithmes peuvent produire des résultats différents

10. Contraintes techniques

Le projet doit respecter les contraintes suivantes :

- langage : Java ;
- application console ;
- une seule classe Java principale ;
- plusieurs méthodes obligatoires ;
- utilisation de tableaux ;
- aucune interface graphique ;
- aucune base de données ;
- aucune bibliothèque externe ;
- ne pas utiliser ArrayList, HashMap ou une structure équivalente pour remplacer l'implémentation algorithmique demandée ;
- les structures nécessaires à BFS/DFS doivent être implémentées par l'étudiant.

Le code doit être compilable et exécutable depuis la ligne de commande.

11. Méthodes attendues

Le découpage exact est laissé à l'étudiant, mais le programme devrait notamment comporter des méthodes permettant de :

- **afficherGrille()**
- **deplacerJoueur()**
- **deplacementValide()**

- **peutPousser()**
- **deplacerCaisse()**
- **partieTerminee()**
- **chargerNiveau()**
- **jouer()**
- **resoudreDFS()**
- **resoudreBFS()**

Les méthodes doivent avoir une responsabilité claire.

12. Niveau standard

Pour valider le projet, les fonctionnalités suivantes sont obligatoires :

- **affichage d'une grille ;**
- **déplacement du joueur ;**
- **déplacement des caisses ;**
- **gestion des murs ;**
- **détection de la victoire ;**
- **plusieurs niveaux ;**
- **validation des entrées ;**
- **compteur de mouvements ;**
- **méthodes correctement utilisées.**

13. Niveau avancé

Pour les étudiants dispensés du PASS Java :

- **résolution automatique par DFS ;**
- **résolution automatique par BFS ;**
- **détection des états déjà visités ;**
- **affichage de la solution ;**
- **comparaison DFS/BFS ;**
- **comptage des états explorés ;**
- **mesure du nombre de mouvements.**

14. Fonctionnalités bonus

Les fonctionnalités suivantes sont facultatives. Elles permettent aux étudiants qui le souhaitent d'aller plus loin dans la réalisation du projet. Les fonctionnalités bonus sont différentes selon le niveau du projet.

Bonus — Niveau standard

Les étudiants du niveau standard peuvent réaliser les fonctionnalités suivantes :

- **Ajout de niveaux supplémentaires.**

- **Redémarrage d'une partie** : permettre au joueur de recommencer le niveau actuel sans relancer le programme.
- **Annulation du dernier mouvement.**
- **Détection de situations manifestement bloquées** : détecter certaines situations simples dans lesquelles une caisse ne peut plus atteindre un objectif.

Bonus — Niveau avancé

Les étudiants du niveau avancé peuvent réaliser les fonctionnalités suivantes :

- **Détection de situations manifestement bloquées**
- **Optimisation du solveur DFS/BFS** : améliorer les performances ou réduire le nombre d'états explorés.
- **Comparaison du temps d'exécution** : mesurer et comparer le temps nécessaire à DFS et BFS pour résoudre une même grille.
- **Recherche d'une solution minimisant le nombre de mouvements.**

Règle d'attribution des bonus

Les fonctionnalités bonus sont facultatives et ne peuvent pas compenser l'absence d'une **fonctionnalité obligatoire** du niveau concerné. Les points bonus sont accordés uniquement si la fonctionnalité est correctement implémentée, fonctionnelle et intégrée au projet. La note finale du projet est plafonnée à 20/20.

15. Exemple de menu final

=====

SOKOBAN JAVA

=====

1. Nouvelle partie
2. Choisir un niveau
3. Résoudre automatiquement
4. Comparer DFS / BFS
5. Afficher les règles
6. Quitter

Votre choix :